

Experiment 15 — Option pricing: the positive part had no spelling

Mathilda — execution-speed experiments

Contents

Experiment 15 — Option pricing: the positive part had no spelling	1
Hypothesis	1
The kernels	2
The finding, before any timing: it could not be written	2
The second finding: one unpacked value in the setup	2
The third: Join's boundary lists	3
What the threshold crossing actually costs	3
Results	3
What is still open	3
Why Mathilda is not the fastest here, and what it would take	3
Verification	4

Experiment 15 — Option pricing: the positive part had no spelling

Date: 2026-07-31 · Code: `src/piecewise.c`, `src/ndkernels.c`, `src/core.c`, `src/ndstruct.c`, `src/info.c` · Result: binomial tree 13.65 s → 148 ms (21.9× faster than Mathematica); explicit finite difference 42.60 s → 316 ms (35.5×)

Common method in [README.md](#).

Hypothesis

Two things no benchmark in the suite had done.

1. A working vector that shrinks. A binomial tree starts with 4001 nodes and ends with 1, so its last few hundred iterations run below `PACK_MIN_ELEMENTS`. Every packed-array benchmark so far has been comfortably above the threshold or comfortably below it; none has crossed it inside a loop. That is the boundary the whole design is built around and it had never been measured.

2. An early-exercise projection. An American option is priced by taking, at every node of every time step, `max(continuation, payoff)`. That is the positive part — the single most common nonlinearity in numerical code, and the same operation as a rectified linear unit.

Two independently-written pricers are used so they check each other: a 4000-step Cox-Ross-Rubinstein tree and a 1000×25000 explicit finite-difference solver in log-price. They agree to 0.6%, which is the right level of agreement for those two discretisations and a much stronger statement than either agreeing with itself across three systems.

The kernels

id	kernel	what it stresses
amt tree	CRR American put, 4000 steps	a shrinking vector across the packing threshold
amfd	explicit FD American put, 1000×25000	25000 iterations of a stencil with boundaries
mcvar	Monte-Carlo VaR, 250000×64	dgemv, Sort, a tail mean

The finding, before any timing: it could not be written

The positive part has three standard spellings. In Mathilda:

spelling	result
<code>Ramp[x]</code>	did not exist
<code>Clip[x, {0., Infinity}]</code>	returned unevaluated — the infinite bound failed to parse
<code>x UnitStep[x]</code>	worked: two passes, and a mixed Real/Integer product

So the only working spelling of `max(x, 0)` over an array was the one nobody writes, and it is the one this benchmark had to be written with to produce a baseline at all.

Both gaps were closed:

- **Ramp** is a new builtin — `Listable`, `NumericFunction`, `Protected`, matching Mathematica's attributes — with a threaded buffer kernel. Its exactness rule is what makes the kernel simple: the zero returned for a negative argument carries the argument's exactness (`Ramp[-1.]` is `0.`, `Ramp[-3]` is the exact `0`), so a `Real` buffer maps to a `Real` buffer and there is nothing to gate. `Clip` does not have that property — it returns the bound at a clipped position — which is why `Clip` needs a gate on its bounds and `Ramp` needs none.
- **Clip** with an infinite bound now works in both the scalar and the buffer path. An infinite bound is never attained, so it cannot put its own head into the answer and is exempt from the exactness gate; a finite exact bound beside `Real` data still is not.

10 ⁶ float64		before	after
<code>Ramp[v]</code>	— (did not exist)		1.41 ms
<code>Clip[v, {0., Infinity}]</code>	— (unevaluated)		0.93 ms
<code>v UnitStep[v]</code> — the workaround		14.5 ms	14.5 ms

`Ramp` is now 10× faster than the only spelling that previously worked.

The second finding: one unpacked value in the setup

The finite-difference row went from 42.60 s to 1.66 s from the `Ramp` change alone — and `Ramp` is not called in its loop. It is called once, in the setup, to build the payoff vector:

```
fdpay = Ramp[K - S0 Exp[x]];          (* once *)
...
While[k < 25000, ...; v = MapThread[Max, {v, fdpay}]; ...]
```

With `Ramp` unpacked, `fdpay` was a plain `List`. `MapThread[Max, {v, fdpay}]` then has one packed operand and one plain one, the pair cannot be absorbed into a rank-2 buffer, and `v` is materialised — 25000 times, once per time step, because of a single value built before the loop started.

A value's representation is a property of its whole lifetime, not of the expression that produced it. The cost of leaving one vector unpacked is paid by every iteration that meets it.

This is the third sweep's "the largest win was a return statement" with the arrow reversed: there, a callee unpacked its results and the caller paid; here, the setup unpacked one value and the loop paid.

The third: **Join**'s boundary lists

What remained of the FD row after that was its boundary handling:

```
v = Join[{lo}, interior, {hi}]
```

`ndstruct_join` required every operand to be a buffer, so a two-element boundary list beside a 998-element interior sent the whole call down the List path and materialised the interior. Packing the small operands up — the rule already applied at Dot, Outer and the Listable gate — took the row from 1.66 s to 316 ms.

The lifted operand must sniff to the same dtype and is never coerced: `Join[{1}, realBuffer]` is a mixed exact/inexact answer in Mathematica, so the exact 1 sniffs to `int64`, fails the dtype test, and the whole call declines to the List path, which gives the mixed answer.

What the threshold crossing actually costs

The binomial tree's hypothesis — that the last few hundred iterations fall off the buffer — turned out to be true and not to matter. At 148 ms for 4000 iterations the row averages 37 μ s per step, and the sub-threshold tail is a few hundred iterations over vectors of a few hundred elements: under 2 ms in total. The threshold is a cliff in ratio and a rounding error in absolute time, because by the time you are below it there is nothing left to compute.

Results

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
Binomial American put, 4000 steps	13.65 s	148 ms	3.248 s	44.7 ms	21.9× faster than WL
Explicit FD American put, 1000 × 25000	42.60 s	316 ms	11.202 s	204 ms	35.5× faster than WL
Monte-Carlo VaR, 250000 × 64	12.9 ms	12.3 ms	22.7 ms	7.5 ms	1.85× faster than WL

92× and 135×, and all three rows are now ahead of Mathematica. The two pricers answer 8.6745 and 8.7257 in all three systems.

What is still open

- 3.3× NumPy on the tree and 1.55× on the FD solver are per-iteration dispatch: 4000 and 25000 interpreter iterations, each doing five or six array operations on a few thousand elements. That is the regime where the constant cost of an evaluation step is comparable to the work in it, and it is the same ceiling experiment 18 runs into head-on.
- **Ramp** on an integer buffer materialises. `to_int` in the unary-kernel descriptor means real-in/integer-out, which is a different function; an integer Ramp needs the same treatment Abs got in experiment 14.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
Binomial tree, 4000 steps	148 ms	44.7 ms (NumPy)	3.32×	4000 iterations × 6 array ops, each on a shrinking vector
Explicit FD, 1000 × 25000	316 ms	204 ms (NumPy)	1.55×	25000 iterations × 6 array ops on 1000 elements
Monte-Carlo VaR	12.3 ms	7.5 ms (NumPy)	1.63×	one dgemv plus a Sort

All three are ahead of Mathematica — by 21.9×, 35.5× and 1.85× — and behind NumPy by a factor that shrinks as the arrays grow. That is the signature of per-operation overhead, not of any kernel being slow.

Arithmetic: the FD row is 25000 steps $\times$ 6 operations = 150000 array operations in 316 ms, or 2.1 μ s per operation on 1000 float64. The memory traffic for one such operation is 8 KB, which at any plausible bandwidth is under 0.5 μ s. So roughly three quarters of this row is the cost of getting to the kernel: an evaluate_step, an attribute read, a packed-argument gate check, an allocation, a refcount.

The road to fastest

1. Cut the per-operation constant (plan 5.2). This is now the dominant term in three separate experiments — 15, 16 and 18 — and no further packing work will move any of them. It wants a profile rather than a hypothesis: the candidates are the SymbolDef lookup and attribute read on every call, the packed-argument scan, and the per-result allocation. At 1000-element arrays every 0.5 μ s removed is worth ~15% of this row.
2. Fuse the stencil sweep. a Most[Most[v]] + b Take[v, {2, -2}] + c Rest[Rest[v]] is five passes and four temporaries to produce one vector. A fused three-point stencil kernel — which Compile[] already generates, and which ordinary array code cannot reach (plan 9.2) — makes it one pass with no temporary. Expected: the FD row to roughly 120 ms, ahead of NumPy.
3. A cheap **Take** with a unit-step span. Take[v, {2, -2}] still builds a position array per axis before the block copy (plan 9.5). A start/step/n selector makes it a memcpy; the same change closes experiment 12's gather.
4. For the tree specifically: let the shrinking vector stay packed below the threshold. Once a value is a buffer, unpacking it because it has dropped under 250 elements is pure loss — the packing decision belongs at creation, not at every operation. The tail of the tree is only ~2 ms, so this is small here, but it is the same rule as experiment 18's, where it is worth 20 $\times$.

Items 1 and 2 put all three rows ahead of NumPy. The tree's 3.3 $\times$ is the hardest of the three, because 4000 iterations of six operations is 24000 dispatches no matter how fast each one is.

Verification

- ~30 differential cases for Ramp (real, integer, Rational, symbolic, complex, matrix, decidable-symbolic, and the packed/unpacked pair), and the Mathematica 14.0 reference values for each read off directly and asserted.
- Clip with {0., Infinity}, {-Infinity, 1.}, {-Infinity, Infinity} and the exact-bound-beside-Real case, packed and plain.
- Join with small plain operands at rank 1 and 2, the dtype-mismatch decline, the symbolic decline and the Join[a, b, level] form.
- A composition case that runs the explicit-FD sweep itself, packed against unpacked.